Home / ASP.NET Core Web API Tutorial

304 HTTP Status Code in ASP.NET Core Web API

304 HTTP Status Code in ASP.NET Core Web API

In this article, I will discuss how to **Return the 304 Not Modified HTTP Status Code from the ASP.NET Core Web API** in ASP.NET Core Web API Tutorial. Please read our previous article on How to Return **100 HTTP Status Code** in ASP.NET Core Web API with Examples.

What is the 304 HTTP Status Code?

The HTTP 304 status code, also known as "Not Modified," is used in web communication to indicate that the requested resource has not been modified since the last time it was requested. The response is typically sent with conditional HTTP headers to ensure efficient caching and bandwidth management. For a guide of the HTTP status codes.

How 304 HTTP Status Code Works?

- **Initial Request:** When a client (such as a web browser) requests a resource, it can include headers like **If-Modified-Since** or **If-None-Match** in the request. These headers contain information about the last version of the resource the client has cached.
- **Server Response:** The server checks if the requested resource has changed since the last request. If the resource hasn't been modified, the server responds with a 304 Not Modified status code, indicating the cached version is still valid. The Response body contains no data, and the client can use the cached version of the resource.
- **Client Action:** Upon receiving the 304 response, the client knows to use the version of the resource it already has in its cache rather than downloading it again.

Benefits of Using 304 Not Modified

This mechanism is particularly useful in reducing bandwidth usage and improves performance by avoiding unnecessary data transfer. The server and client exchange only the minimal information needed to confirm the cache's validity.

- **Bandwidth Savings:** Since the server does not send the resource again, it saves bandwidth.
- **Faster Load Times:** The client can load the cached version of the resource much faster.
- **Reduced Server Load:** The server does not need to process and send resources that have remained the same, reducing its load.

How to Implement 304 HTTP Status Code in ASP.NET Core Web API?

Implementing the HTTP status code in an ASP.NET Core Web API involves using caching mechanisms and setting appropriate headers to allow the client to make conditional requests. The following are steps to implement a 304 HTTP status code:

- Enable Response Caching Middleware
- Set Cache-Control Headers in the Response
- Configure Response Caching in the Controller
- Use Conditional Headers in Client Requests

Enable Response Caching Middleware

First, ensure that response caching is enabled in your application. This typically involves adding the response caching middleware. Here, we are adding a Response Caching service to the built-in dependency injection container and a Response Caching middleware component to the Request Processing Pipeline.

```
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddControllers();
builder.Services.AddResponseCaching();
var app = builder.Build();
app.UseResponseCaching();
app.MapControllers();
app.Run();
```

Set Response Caching Headers in the Controller

Next, modify the controller to set the cache headers, including the response caching middleware. Here, we are adding a Response Caching service to the built-in dependency injection container and a Response Caching middleware component to the Request Processing Pipeline.

```
using System.Text;
using System.Threading.Tasks;
namespace Controllers
{
    public class ProductsController : ControllerBase
    {
        [HttpGet("{id}")]
        [ResponseCache(Duration = 60, Location = ResponseCacheLocation.Client, NoStore = false)]
        public IActionResult GetProduct(int id, [FromQuery] bool? forceUpdate)
        {
            // Simulating a product retrieval
            var product = new { Id = id, Name = "Example Product", Price = 100 };
            var etag = "\"" + id.ToHashCode().ToString() + "\"";
            var ifNoneMatch = Request.Headers["If-None-Match"];
            if (!forceUpdate.HasValue && ifNoneMatch == etag)
            {
                return StatusCode(304);
            }
            Response.Header.Add("ETag", etag);
            return Ok(product);
        }
        [HttpPost]
        [ResponseCache(NoStore = true)]
        public IActionResult CreateProduct([FromBody] Product product)
        {
            // Simulating product creation
            return Ok("Product created");
        }
        [HttpPut("{id}")]
        [ResponseCache(NoStore = true)]
        public IActionResult UpdateProduct(int id, [FromBody] Product product)
        {
            // Simulating product update
            return Ok("Product updated");
        }
        [HttpDelete("{id}")]
        [ResponseCache(NoStore = true)]
        public IActionResult DeleteProduct(int id)
        {
            // Simulating product deletion
            return Ok("Product deleted");
        }
    }
}
```

Configure Response Caching in the Controller

Now, modify the controller to set the cache headers, including the response caching middleware. Here, we are adding a Response Caching service to the built-in dependency injection container and a Response Caching middleware component to the Request Processing Pipeline.

```
using System.Text;
using System.Threading.Tasks;
namespace Controllers
{
    public class ProductsController : ControllerBase
    {
        [HttpGet("{id}")]
        [ResponseCache(Duration = 60, Location = ResponseCacheLocation.Client, NoStore = false)]
        public IActionResult GetProduct(int id, [FromQuery] bool? forceUpdate)
        {
            // Simulating a product retrieval
            var product = new { Id = id, Name = "Example Product", Price = 100 };
            var etag = "\"" + id.ToHashCode().ToString() + "\"";
            var ifNoneMatch = Request.Headers["If-None-Match"];
            if (!forceUpdate.HasValue && ifNoneMatch == etag)
            {
                return StatusCode(304);
            }
            Response.Header.Add("ETag", etag);
            return Ok(product);
        }
        [HttpPost]
        [ResponseCache(NoStore = true)]
        public IActionResult CreateProduct([FromBody] Product product)
        {
            // Simulating product creation
            return Ok("Product created");
        }
        [HttpPut("{id}")]
        [ResponseCache(NoStore = true)]
        public IActionResult UpdateProduct(int id, [FromBody] Product product)
        {
            // Simulating product update
            return Ok("Product updated");
        }
        [HttpDelete("{id}")]
        [ResponseCache(NoStore = true)]
        public IActionResult DeleteProduct(int id)
        {
            // Simulating product deletion
            return Ok("Product deleted");
        }
    }
}
```

Here,

- The Get method retrieves a resource by its identifier and sets appropriate caching headers.
- The Post method and other methods specify that the response can be cached for 60 seconds.
- The Put and Delete methods specify that the response should not be cached.
- If the resource has not changed (indicated by matching ETags), the HTTP 304 Not Modified status is returned.
- The ifNoneMatch variable retrieves the client's cached ETag from the request header.

Use Etag and Last-Modified Headers

The above example demonstrates how to use the Etag and Last-Modified headers:

- **Etag Header:** A unique identifier for the version of the resource.
- **Last-Modified Header:** The date and time when the resource was last modified.

Testing the Implementation

Initial Request

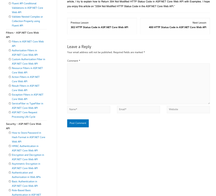
The client makes an initial request to get the resource. Let us see the Http.get request.

- Open Postman
- Create a new GET request
- Set the URL to the endpoint, e.g., https://localhost:7247/api/products/1
- Set the request, and you should get a 200 OK Response with the response body stored in the image below.

Subsequent Request with If-None-Match Header

Now, make a request for the same resource. The server checks the If-None-Match header and responds with 304 Not Modified if the resource has not changed.

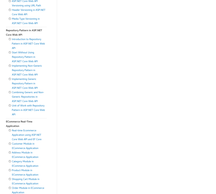
- Create a new GET request
- Set the URL to the same endpoint, e.g., https://localhost:7247/api/products/1
- Create a new GET request
- Set the URL to the same endpoint, e.g., https://localhost:7247/api/products/1
- Add a new header with the key If-None-Match and the value of the ETag copied from the initial response.
- Send the request.



Subsequent Request with If-Modified-Since Header

Make a request for the same resource with the If-Modified-Since header.

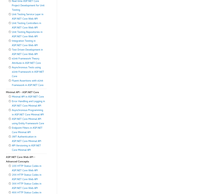
- Create a new GET request
- Set the URL to the same endpoint, e.g., https://localhost:7247/api/products/1
- Create a new GET request
- Set the URL to the same endpoint, e.g., https://localhost:7247/api/products/1
- Add a new header with the key If-Modified-Since and the value of the Last-Modified header copied from the initial response.
- Send the request.



Subsequent Request with If-Modified-Since Header

Make a request for the same resource with the If-Modified-Since header.

- Create a new GET request
- Set the URL to the same endpoint, e.g., https://localhost:7247/api/products/1
- Create a new GET request
- Set the URL to the same endpoint, e.g., https://localhost:7247/api/products/1
- Add a new header with the key If-Modified-Since and the value of the Last-Modified header copied from the initial response.
- Send the request.



When Should We Use the 304 HTTP Status Code in ASP.NET Core Web API?

The 304 HTTP status code in ASP.NET Core Web API Tutorial is useful in various scenarios:

- **Bandwidth Optimization:** When a client requests a resource that has not changed since the last request, the server can respond with a 304 Not Modified status code, indicating that the client can use the cached version of the resource. This reduces the amount of data transferred over the network, saving bandwidth and improving load times.
- **Improving Client Performance:** This is ideal for scenarios where bandwidth is limited or expensive, such as mobile networks. The client can quickly load the resource from its cache without waiting for the server to send it.
- **Reducing Server Load:** The server does not need to process and send the resource again, which can be beneficial for high-traffic websites.
- **Enhancing User Experience:** Optimizing bandwidth usage by preventing the re-transfer of unchanged data is especially important for users with limited bandwidth.

In the next article, I will discuss how to Return **400 HTTP Status Code in ASP.NET Core Web API** with Examples. In this article, I try to explain how to Return **304 Not Modified HTTP Status Code in ASP.NET Core Web API** with Examples. I hope you enjoy the article on 304 Not Modified HTTP Status Code in ASP.NET Core Web API.

Previous article
400 HTTP Status Code in ASP.NET Core Web API
Next article
405 HTTP Status Code in ASP.NET Core Web API

Leave a Reply

Your email address will not be published. Required fields are marked *

Comment *

Name *
Email *
Website

Post Comment

Home Privacy Policy Contact ASP.NET Tutorial Angular Tutorials ASP.NET Core Blazor Tutorials ASP.NET Core Tutorials ASP.NET Web Forms Tutorials ASP.NET Web API Tutorials Azure Tutorials Cloud Computing Tutorials Data Structure Tutorials Design Patterns Tutorials Dotnet Interview Questions and Answers Core Java Tutorials Entity Framework Tutorials Interview Questions and Answers Java Tutorials LINQ Tutorials Python Tutorials SQL Server Tutorials SQL Server Interview Questions and Answers Tutorial Gateway Tutorials Visual Studio Tutorials Web Technology Tutorials
Copyright © 2024 Dot Net Tutorials. All Rights Reserved.
Powered by Dot Net Tutorials
Dot Net Tutorials is a Microsoft Partner